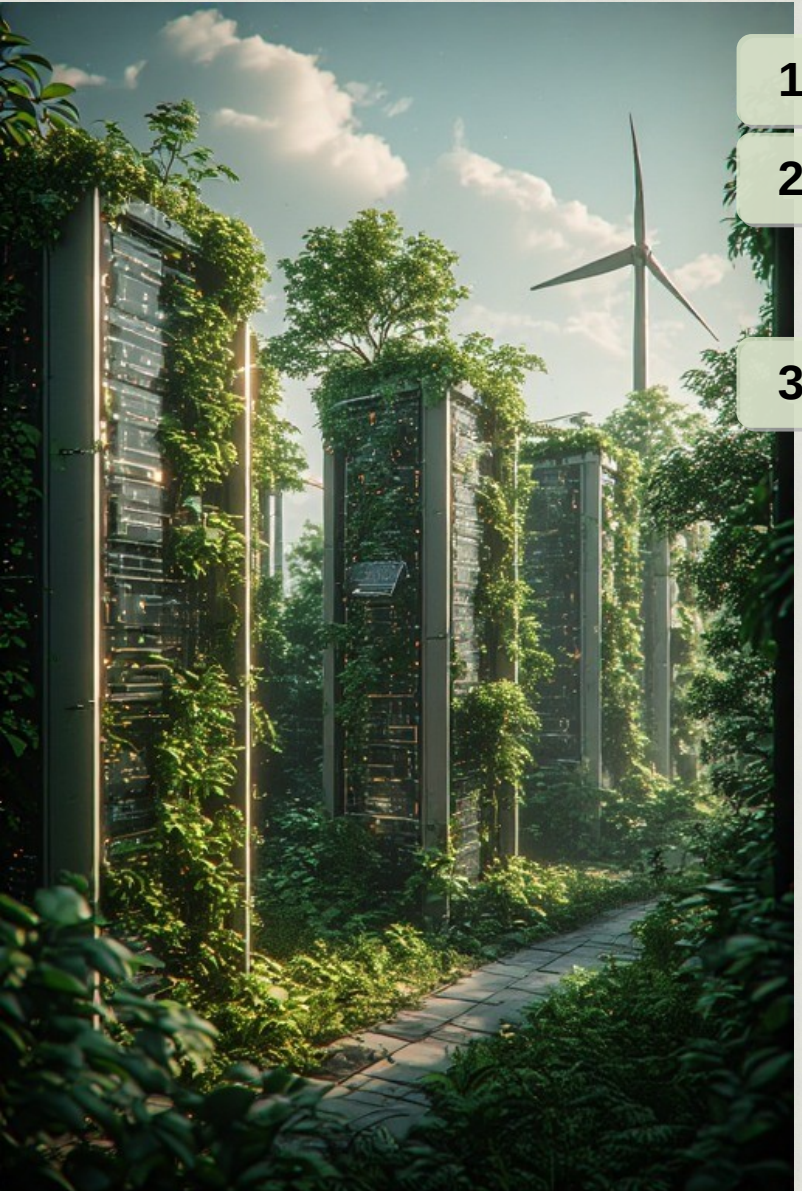
Sistemas Paralelos
Explicación práctica
Principio de localidad aplicado

Universidad Nacional de La Plata

Facultad de Informática

Prof. Adrian Pousa (apousa@lidi.info.unlp.edu.ar)



1 **Principio de localidad**

2 **Memoria cache**

Latencia de componentes hardware. Memoria cache y jerarquía de memoria. Contención y coherencia. Mitigación de contención y coherencia en software.

3 **Principio de localidad aplicado**

Multiplicación de matrices cuadradas.

1 Principio de localidad

Principio de localidad

Temas presentados en “Clase 2 – Sistema de memoria” y aplicados en la práctica.

- **Principio de localidad [*Peter J. Denning (1968)**]:** Los accesos a memoria de un determinado programa no se distribuyen de manera uniforme por todo el espacio de direcciones, sino que tienden a concentrarse temporalmente en posiciones cercanas.
- Se manifiesta en dos formas:
  - **Localidad temporal:** Un dato consultado hace poco es muy probable que sea utilizado nuevamente en un futuro próximo.
  - **Localidad espacial:** Al acceder a un dato, es muy probable que otros datos ubicados en direcciones contiguas se necesiten en un futuro próximo.

**The Working Set Model for Program Behavior*

2

Memoria cache

Latencia de componentes hardware. Memoria cache y jerarquía de memoria. Contención y coherencia. Mitigación de contención y coherencia en software.

Memoria cache
Latencia de componentes hardware

Evento	Tiempo Real	Tiempo Escalado
1 ciclo de CPU	0.3 ns	1 s
Acceso a cache de Nivel 1	0.9 ns	3 s
Acceso a cache de Nivel 2	2.8 ns	9 s
Acceso a cache de Nivel 3	12.9 ns	43 s
Acceso a RAM	120 ns	6 min
E/S a SSD	50-150 µs	2-6 días
E/S a HDD	1-10 ms	1-12 meses
Internet: San Francisco a Nueva York	40 ms	4 años
Internet: San Francisco a UK	81 ms	8 años
Internet: San Francisco a Australia	183 ms	19 años
Reboot de SO de VM	4 s	423 años
Reboot de SO físico	5 m	32 milenios

Memory Wall (Slide 4 – Clase 2)

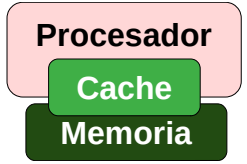
La velocidad de la CPU creció más rápido que la de la memoria.

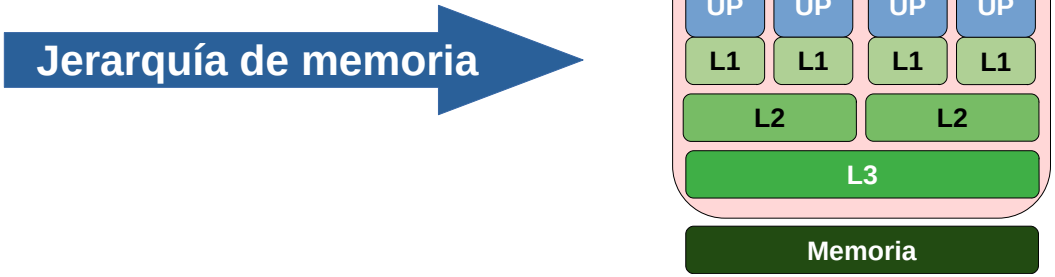
La CPU pasa la mayor parte de su tiempo esperando datos.

Libro: Systems Performance - Brendan Gregg

Memoria cache y jerarquía de memoria

- El principio de localidad es la base para entender las memorias caché.
- Memoria cache (***Slides 9 y 10 – Clase 2***):
  - Memorias de alta velocidad y baja capacidad integradas al chip.
  - Memoria intermedia entre los registros de la CPU y la memoria principal.
  - **Objetivo:** disminuir la latencia del sistema de memoria maximizando el número de datos que se acceden desde de la caché.





- Explotar localidad espacial y temporal de datos es crítico para amortizar la latencia e incrementar el ancho de banda efectivo (***Slide 18 – Clase 2***)

Memoria cache
Contención y coherencia

- Explotar **localidad** espacial y temporal de datos es crítico para amortizar la latencia e incrementar el ancho de banda efectivo (***Slide 18 – Clase 2***)
- Además de la localidad, necesitamos considerar otros aspectos relacionados a caché:
  - **Contención:** Conflicto en acceso a un recurso compartido (secuencial).
  - **Coherencia** (**Slide 30 – Clase 2**):
    - False sharing

Como programadores…
¿Cómo podemos utilizar adecuadamente la memoria caché para mejorar el rendimiento?

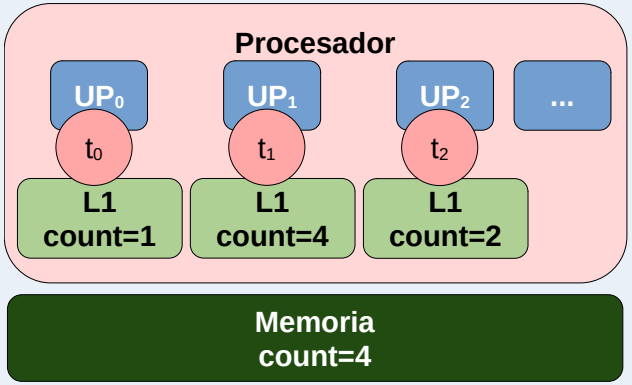
Memoria cache
Mitigación de contención y coherencia en software

- **Problema (Práctica 1 de Programación Concurrente):** Dado un número N calcular cuántas veces aparece ese número en un arreglo v de longitud M. Paralelizar con H hilos sobre H unidades de procesamiento. (M proporcional a H)

```
count = 0
Array v[M]
```

```
Hilo t[id:0 to H-1]{
 Part = M/H
 Ini = id*Part
 End = Ini + Part - 1

   for i = Ini to End
     if (v[i]==N)
      atomic(count=count+1)
}
```



Peor caso:
Toda posición del arreglo es N.
M accesos de escritura a la misma posición de memoria.
Se secuencializa (**contención alta**)
Protocolo de coherencia 100% activo.

```
Hilo t[id:0 to H-1]{
 local = 0
 Part = M/H
 Ini = id*Part
 End = Ini + Part - 1

   for i = Ini to End
     if (v[i]==N)
      local=local+1
   atomic(count=count+local)
}
```

Solución: Contar en una variable local minimizando accesos a la variable compartida (**contención baja**).

Mitigación de contención y coherencia en software

- **Empíricamente:**
  - Implementamos el algoritmo secuencial y paralelo en pthreads.
  - Ejecutamos el peor caso para una carga de trabajo de 2^{30} elementos enteros.
  - Pthreads ejecutado con 16 hilos en 16 unidades de procesamiento.
  - Tiempos de ejecución medidos en segundos.

Secuencial	Paralelo Variable compartida	Paralelo Variable local
3,41s	**140,35s**	**0,71s**

Peor que secuencial!!!

Cuando el objetivo es mejorar el rendimiento, no debemos abstraernos completamente del hardware.

3

Principio de localidad aplicado

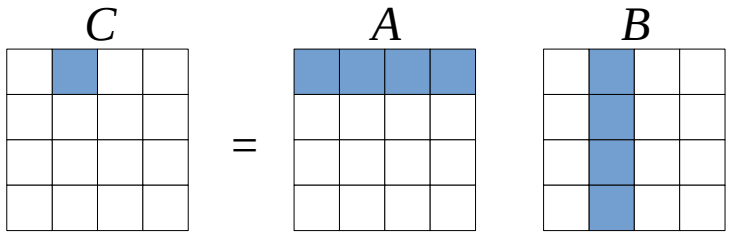
Multiplicación de matrices cuadradas.

Principio de localidad aplicado
Multiplicación de matrices cuadradas

- Multiplicación de dos matrices cuadradas de NxN:

$$C = AB$$

$$c_{i,j} = \sum_{k=0}^{N-1} a_{i,k} \cdot b_{k,j}$$



- Por la naturaleza del patrón de acceso y para aprovechar la localidad, almacenamos en memoria por filas (*row-major*) las matrices A y C, y por columnas (*column-major*) la matriz B.

Pseudocódigo

```
for i = 0 to N-1
 for j = 0 to N-1
     C[i,j] = 0
     for k = 0 to N-1
          C[i,j] = C[i,j] + A[i,k]*B[k,j]
```

```
for i = 0 to N-1
 for j = 0 to N-1
  for k = 0 to N-1
   C[i*N+j] = C[i*N+j] + A[i*N+k]*B[k+j*N]
```

¿Por qué?

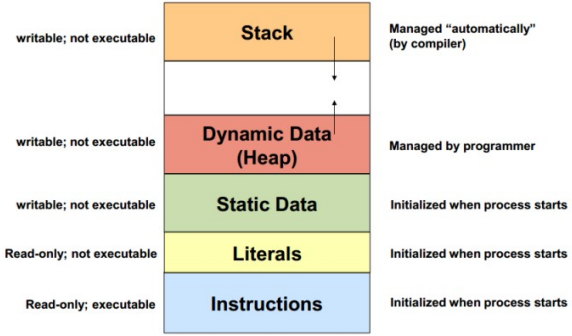
(Slide 19 – Clase 2)

Principio de localidad aplicado
Multiplicación de matrices cuadradas

- **Declaración de matrices en le lenguaje C**:

```
int A [N][N];
int B [N][N];
int C [N][N];
```

- **Desventajas**:
  - Se almacenan en memoria estática (Pila o *Stack*).
  - Tamaño de matrices limitado por la Pila.
    - Posibilidad de *stack overflow*.



- **Mejor definirlas**:
- **Ventajas**:
  - Mejor control de almacenamiento.
  - Se almacenan en memoria dinámica (*Heap*).
  - Tamaño de matrices eventualmente "ilimitado".

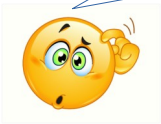
```
tipo* A;
tipo* B;
tipo* C;
…
A= (tipo*)malloc(sizeof(tipo)*N*N);
B= (tipo*)malloc(sizeof(tipo)*N*N);
C= (tipo*)malloc(sizeof(tipo)*N*N);
…
free(A);
free(B);
free(C);
```

Principio de localidad aplicado
Multiplicación de matrices cuadradas

¿Cómo acceder al elemento *(x,y)* de una matriz M?

- Si la matriz está almacenada por filas: `Mf[x*N+y];`
- Si la matriz está almacenada por columnas: `Mc[x+y*N];`

Quiero acceder a la posición *(i,j)* *de una matriz A*?



¿Cómo está almacenada en memoria la *matriz A*?

***A* por filas!!!**

$$Mf_{x,y} = Mf[x \cdot N + y]$$

$$A_{i,j} = A[i \cdot N + j]$$

***A* por Columnas!!!**

$$Mc_{x,y} = Mc[x + y \cdot N]$$

$$A_{i,j} = A[i + j \cdot N]$$

Principio de localidad aplicado
Multiplicación de matrices cuadradas

- Ejemplo: Comparación de dos algoritmos secuenciales, ambos $O(n^3)$:
  - Matrices A, **B** y C almacenadas en memoria por **filas**.
  - La matriz **B** almacenada en memoria por **columnas**.
- Características de la ejecución:
  - Tamaños de matrices N= 512, 1024, 2048
  - Tiempos de ejecución medidos en segundos
  - Medimos fallos de cache de nivel 1:
    - *PAPI (Performance Application Programming Interface)*

	B por filas		B por columnas	
N	Tiempo	Fallos L1	Tiempo	Fallos L1
512	1,60s	135.411.031	1,08s	16.872.076
1024	26,26s	1.097.641.458	8,87s	135.070.148
2048	265,59s	14.005.385.456	71,28s	1.157.777.331

Mas fallos de caché peor tiempo de ejecución

Principio de localidad aplicado
Multiplicación de matrices cuadradas

¿Y si multiplicamos C = AA?

- Dos opciones (ambas $O(n^3)$):
  - Multiplicamos AA tal como está, **A** almacenada en memoria por **filas**.
  - Creamos una matriz auxiliar $\mathbf{A_{col}}$ almacenada en memoria por **columnas**:
    - Mas consumo de memoria
    - Mas operaciones/instrucciones ¿Mas tiempo de ejecución?

N	C=AA	$C=AA_{col}$
512	**Tiempo:**1,76s **Fallos L1:**134.637.258 **Instrucciones:**6.176.380.648	**Tiempo:**1,25s **Fallos L1:**17.172.672 **Instrucciones:**6.182.676.494
1024	**Tiempo:** 19,91s **Fallos L1:**1.078.625.820 **Instrucciones:**49.401.576.181	**Tiempo:** 10,09s **Fallos L1:**136.042.686 **Instrucciones:**49.426.748.663
2048	**Tiempo:**293,36s **Fallos L1:**11.588.145.094 **Instrucciones:**395.174.834.418	**Tiempo:**89,59s **Fallos L1:**1.178.020.959 **Instrucciones:**395.275.469.418

Medimos fallos e instrucciones con PAPI

N=2048, A_{col} ejecuta 100.635.000 de instrucciones adicionales, 10.410.124.135 menos de fallos de cache.

Mas instrucciones, Menos fallos

Mejor rendimiento!!!